Checking or Recomputing? Two Cost Regimes in Verifying AI-Generated Formal Artifacts

Yusheng Zheng
UC Santa Cruz
yzhen165@ucsc.edu

Abstract

A proof term lets a checker validate a result without repeating the search that found it. Proofs by computation retain a different cost: the checker must evaluate the decision procedure embedded in the term. We study this distinction in OpenAI's AI-generated Lean library. For two computational certificates, checking the target takes 80–82% of compilation time, compared with 0.5–7% for two ordinary proofs. The library contains 10,668 times more `decide +kernel` calls per source megabyte than Mathlib. Compiling 162,170 of its 163,096 modules used 169 P-core and 302 E-core CPU-hours. Default build targets omit a third of the computation call sites. Profiles show why checking time alone can also mislead: dependency imports, module discovery and Lean's memory guard can dominate an invocation. We present ProofServe, which rechecks projects in parallel with Lean's original kernel, supports restart, and reports the modules and axioms covered.

1 Introduction

AI systems can produce formal proofs at a scale that makes rechecking an operational concern. A successful compilation produces compiled module files, but a project owner may still want to recheck its proof terms, identify its axioms, or determine which files a default build actually covered. These tasks require knowing both what a checking command does and where its time goes.

The usual reason to expect cheap checking is that a stored proof records the successful steps of a search. The checker validates those steps instead of trying unsuccessful alternatives again. This reasoning changes for a proof by computation. A small proof term may certify a finite calculation by asking the kernel to evaluate it. The term saves the answer's justification, not necessarily the work needed to validate that justification.

We measure both cases in OpenAI's Lean mathematics library. Two computational certificates retain about four-fifths of their compilation time when we recheck their target proof terms. Two ordinary controls retain less than 7%. Yet the presence of these calls does not divide modules into uniformly cheap and expensive classes. A 15.9 KB module without them is the most expensive module in our compilation campaign. We combine these case studies with per-module compilation measurements and profiles of Lean's tooling.

The tooling matters because an invocation does more than check a proof. It locates compiled modules, imports dependencies, builds lookup tables and enforces resource limits. For short modules, these fixed costs can outweigh proof replay. For long computational proofs, repeatedly polling memory usage can itself become a major cost. A faster command may also be checking a different collection of declarations. We make these differences explicit before comparing timings.

We make four contributions:

- We measure when proof checking repeats computation, and relate these cases to the prevalence of kernel-evaluated decisions in an AI-generated library.
- We characterize compilation across 162,170 completed modules, showing cost concentration and gaps in the default build's source coverage.
- We explain import, resource-accounting and module-discovery overheads with existing controls, profiles and small official-checker examples.
- We present ProofServe and evaluate its parallel checking, restart support and external-project coverage through the installed tool.

The measurements motivate a practical rule: name the modules, proof terms and setup work included in a checking result, rather than treating every successful command as the same verification task.

2 Background

Elaboration and kernel checking. Lean elaborates source text and tactics into typed terms. Tactics can search for a proof, while the trusted kernel checks the resulting term and its type [2]. Compilation also loads imports and writes these files. Consequently, compiler time includes much more than the final proof check.

The `decide` mechanism constructs proofs of decidable propositions. With `decide +kernel`, Lean's kernel performs the reduction needed to validate the decision [4].

We call such source occurrences *computation call sites*. We count them after masking comments and strings. Their presence is a useful source label, not a measurement of runtime: ordinary `decide` can also require substantial computation.

What a project check covers. A *root* is a module requested for checking. Its *dependency closure* is the module together with the imports it needs, recursively. We use *replay* to mean checking a stored declaration again. The official `leanchecker` default mode loads dependencies and replays the target module's declarations, trusting imported declarations. Its `-fresh` mode imports the target, merges the imported declarations, and replays the resulting set into an empty kernel environment [3]. Here, *fresh* describes the checking environment, not an empty OS file cache. Both modes use Lean's original kernel and skip unsafe or partial declarations.

A default `lake build` starts from the project's configured default targets and follows their imports. Files outside that reachable set need not be built. Thus compilation coverage, checking a requested root, and rechecking every original proof body are three distinct questions.

3 Method

Corpus and machine. We study the published OpenAI mathematics repository [11], revision `4eaee875`, with Lean 4.34.1 and its pinned Mathlib dependency. It contains 163,096 executable source modules, plus a separate configuration unit. We run on an Intel i9-14900K. The compilation campaign uses eight performance (P) cores and fourteen efficiency (E) cores; selected-case timings and ProofServe evaluation use the eight P cores. We report P- and E-core CPU time separately.

Compilation and coverage. We compile modules in dependency order with eight Lean workers per module. Initially we allow 22 concurrent module jobs, waiting for running work to finish before retrying memory-limited jobs alone. We then admit jobs according to available memory and defer 926 unfinished modules in memory-heavy families and their dependents. We retain earlier successes from those families. A completed module's CPU cost includes imports, elaboration, checking and artifact emission, but excludes separate prerequisite builds and downstream compilation of generated C. The campaign consumes 31.5 hours of measured execution time. Its complete per-module outcomes and execution chronology are in the artifact.

Clocks, counters and controls. We measure wall time separately from user-plus-system CPU time. Across 20 modules on P/E cores with and without background compiler load, total instruction counts vary by a median 1.33%, largely because OS-kernel counts vary. User-mode counts vary much less: median 0.018%, P95 0.14%. We chose user-mode instructions to compare module work across these conditions after seeing this difference. We keep CPU summaries separate by core type. The instruction counts describe process work, not time inside Lean's logical kernel.

We separate setup from target checking with complementary measurements. For 13 modules selected because historical builds took under five seconds, we compare compilation with an import-only control that loads the same dependencies. Separate profiles identify import hotspots. A second experiment compares native memory-guard settings on three modules, with three repetitions per configuration. The main compilation and ProofServe measurements omit `-M`; the guard experiment deliberately varies it. All profiles, failed attempts and measurement overheads remain in the artifact.

4 Findings

4.1 Computational proofs retain most of their checking cost

The two computational certificates are expensive to recheck because their proof terms retain the computation. After preparing dependencies, we time reading, mapping and replaying the target module. This target-only wall time is 80% of compilation for Grid and 82% for Amplitudes, versus 6.5% for Mathlib Nat.Basic and 0.5% for NearPin. The computational targets spend about 255 and 110 seconds in replay; almost all of that target component is inside the kernel replay call.

The difference follows from the proof representation. A tactic can emit a sequence of mathematical steps that avoids repeating its original search. A computational certificate instead uses a theorem such as `of_decide_eq_true`, whose validation reduces a decision procedure over a finite object. A compact wrapper can therefore conceal substantial evaluation. This quantifies, in selected cases, the computational explanation proposed by an earlier report on the same library [10].

Whole-process timings show why the source label alone is insufficient. Across 31 complete checker/compiler pairs, the median CPU ratio is 0.45. The two source groups overlap: their medians are 0.39 without computation calls and 0.50 with them. A Ramsey certificate without such calls also requires expensive checking. Imports and cheap dependencies can dilute an expensive target in a whole-closure average.

4.2 A small fraction of modules consumes a quarter of CPU time

Compilation cost is concentrated in a small fraction of modules (Figure 2). The median module uses 4.0 P-core or 7.5 E-core CPU seconds. Within each core type, the most expensive 1% consume 25.1% and 25.7% of total CPU time,

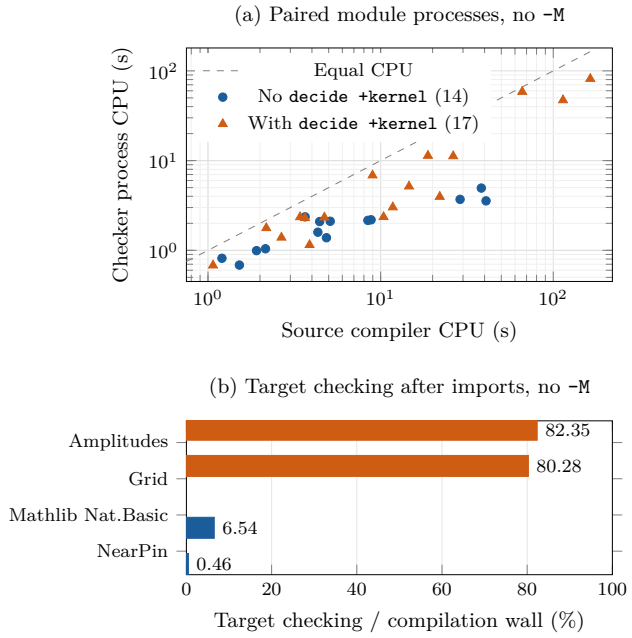


Figure 1: Checking has two useful views. (a) Whole-process CPU for 31 module pairs, grouped by whether source contains `decide +kernel`. (b) Target-only checking wall time after dependency preparation, divided by that target's compilation wall time. The two panels use different clocks.

respectively. The 162,170 successful compilations use 169 P-core and 302 E-core CPU-hours. Across completed modules, median user-mode work is 12.6 billion instructions and the 95th percentile is 78.5 billion.

Larger files tend to cost more, with Spearman rank correlations of 0.62 (P) and 0.60 (E). File size can still misidentify the most expensive cases. The most expensive module, `UltraExpectationFull`, is only 15,860 bytes and contains no computation call sites, yet uses 9,314 E-core CPU seconds. A 5.29 MB Ramsey certificate with 11,114 such sites uses 2,241 E-core CPU seconds. File size and call-site count alone do not identify this extreme cost.

Computation calls nevertheless identify a disproportionately costly subset. The 5,848 modules containing these calls are 3.6% of completed modules and use about 18% of CPU time in each core type. Across the full source tree, we count 1,354,379 computation call sites, versus seven in the pinned Mathlib subtree. After normalizing by source megabytes, the density is 10,668 times higher. These call sites help locate workloads that deserve checking, even though their count is not a complete cost model.

4.3 Default builds omit a third of computation call sites

A successful default build leaves a large part of this library outside its import graph. About 20% of source files are unreachable from the default targets. They contain 451,008

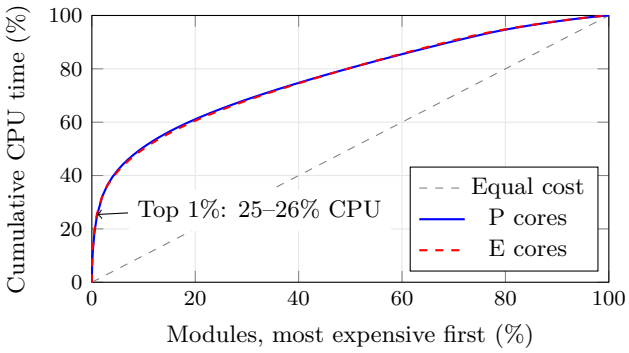


Figure 2: CPU cost accumulates quickly when modules are ordered from most to least expensive within each core type. The top 1% consume about a quarter of CPU time. We keep P and E populations separate.

computation call sites, or 33% of the total. The completed modules outside that graph alone consume about 31 P-core and 56 E-core CPU-hours.

This omission changes the interpretation of a build result. Building all default targets establishes neither that every source file compiles nor that every computational certificate has been checked. The omitted set is not a negligible collection of empty files. An audit should therefore enumerate the source inventory and default-target coverage separately from the modules listed in a checking receipt.

4.4 Imports can dominate compilation of short modules

Dependency loading imposes a cost even when a module adds no proof work. In the 13-module short-build cohort, the import-only wall median is 1.36 seconds and median peak RSS is 3.1 GiB. The import-only control takes a median 68% of the original compilation wall time. The official checker's whole-process wall time is a median 1.28 times compilation time on these modules: checking is sometimes slower than building.

A separate Mathlib import profile explains part of this fixed work. Persistent-object marking, `lean_mark_persistent`, accounts for 9.3% of sampled cycles, and name comparison, `lean_name_eq`, for 5.7%. Import finalization also inserts constants into hash tables and allocates objects through mimalloc. These operations prepare the environment before a target proof can be replayed.

The imports primarily map existing library data: about 93% of proportionally accounted resident memory is file-backed. The 3.1 GiB peak is therefore not 3.1 GiB of newly constructed mutable heap.

Module	Wall		System CPU	
	Guard	No guard	Guard	No guard
Grid	621.936	305.693	436.151	107.412
UMoments9	141.739	88.211	100.597	42.305
EnvelopePsiB	64.447	63.518	2.669	1.960

Table 1: Native memory guard on/off: medians of three runs, seconds. An external 15 GiB memory limit remains in both configurations. Wall and system CPU are different measurements.

4.5 Lean's memory guard can double compilation time

The native `-M` guard is inexpensive on one ordinary module and expensive on two computational workloads. With eight Lean workers, removing it reduces Grid wall time from 622 to 306 seconds and UMoments9 from 142 to 88 seconds. EnvelopePsiB changes only from 64.4 to 63.5 seconds (Table 1). All produced output artifacts agree byte for byte.

Lean polls this guard while reducing and checking terms. With the guard enabled, the pinned executable calls `getrusage` after every 200 memory checks to ask Linux for process resource usage [7]. The Grid baseline records about four million such calls. Its system CPU falls from 436 to 107 seconds without the native guard. Among baseline OS-kernel cycle samples, resource accounting contributes 75% for Grid and 59% for UMoments9; futex synchronization contributes another 20% and 34%. These percentages describe kernel-mode samples, not fractions of wall time. The result explains our choice to omit the native guard in the main campaign while retaining external memory protection.

4.6 Module discovery can dominate an official checker invocation

Even locating a single requested module can be expensive in a large project. We check the same Mathlib Nat.Basic root once with the AI library's large search path and once with a small directory containing hardlinks to the same compiled dependency files. We change the search path, not the proof or its dependencies. Default checking takes 322.9 seconds on the large path, including 234.6 seconds of system CPU, versus 0.394 seconds on the small path.

The official CLI first enumerates candidate artifacts, resolves their module names and then filters for the requested target [3]. A separate discovery-only probe spends 2.6 seconds enumerating the large path and 271.4 seconds resolving names and filtering prefixes. It finds the same one target as the small-path probe. Thus the main observed bottleneck is not merely directory enumeration: per-candidate path resolution can dominate the command before proof replay. This is a search-path-dependent cost, not an intrinsic cost of checking Nat.Basic.

4.7 Checking a merged environment does not check every original body

Official checking modes differ in the artifacts and contexts they inspect. We confirm this with small controlled examples. Removing a compiled module's private fragment leaves default checking successful, while `-fresh` refuses the incomplete input. A separate example explicitly uses `Lean.ofReduceBool`, a native evaluation bridge: default checking succeeds, but fresh replay lacks the required imported code. This bridge is distinct from ordinary `decide +kernel` reduction.

Import merging creates a second boundary. Two modules can define the same theorem interface—name, type and universe parameters—with different proof bodies. The later imported body replaces the earlier one in the merged declaration set. If a deliberately malformed body is overwritten by a good body, `-fresh` checks the surviving body and can succeed. If the malformed body survives, it is rejected. Default checking of an import-only root does not recursively recheck either body.

These examples distinguish checking objectives, not the truth of ordinary source theorems. The missing-fragment and bad-body inputs are deliberately altered serialized artifacts; the explicit native-evaluation example exercises a different supported-context boundary. They do not demonstrate that normal Lean source compilation proves false statements. They do show why *all proofs checked* needs an explicit object: the requested module, the merged environment, or every original module body.

5 ProofServe: design and evaluation

We build ProofServe to make repeated checking usable without replacing Lean's kernel. The `check` command discovers compiled project roots and their dependency closures. It schedules ready declarations against a checked environment, joins worker results, and records completed work for restart. A persistent checked prefix avoids repeatedly preparing the same dependencies. Modules requiring a separate context use a conservative replay path. The `audit` command reports source files, default-target coverage, checked modules and declared axioms.

Proof-body context. By default, ProofServe shares already kernel-checked theorem bodies when their interfaces match. This saves work, but a crafted artifact can depend on the shared body and pass even when official checking in its original import context rejects it. The proposition is still checked using proofs and axioms in the shared closure. The `-strict-theorem-bodies` option isolates differing bodies. On Nat.Basic at eight workers, this costs 4.2 times the default wall time and 2.5 times its CPU time. The corresponding Grid run does not finish within its time budget, giving only a wall-cost lower bound of 5.1 times default.

Closure	1 worker	8 workers	Wall gain
Nat.Basic	110.2/110.8	113.5/65.3	1.70×
Grid	751.2/754.0	835.8/352.3	2.14×
Barrier	732.6/735.6	766.3/425.9	1.73×

Project	N/O modules	ProofServe	Official
Newspeak	754/105	109.1/110.6	38.6/38.7
Solar	2368/12	167.2/169.3	5.5/5.5
AI proofs	10753/9	2481.0/2579.7	37.4/41.0

Table 2: Installed ProofServe results, CPU/wall seconds. Top: medians of three complete runs; Nat.Basic/Grid/Barrier closures have 1,330/3,793/4,449 modules. Bottom: one run per arm, one worker; N/O counts are native closure modules / official local roots. Some other configurations have only one run; see artifact. Unequal native/official obligations do not define a speedup comparison.

Parallel replay. We evaluate the installed tool with fresh checkpoint state and three complete repetitions at each reported scaling point (Table 2). One to eight workers reduces wall time by 1.70, 2.14 and 1.73 times on Nat.Basic, Grid and Barrier. CPU rises by 2.9%, 11.3% and 4.6%. The speedups are useful but modest; more workers do not remove sequential preparation or all coordination costs. Disabling persistent state adds 3–5% wall time on Nat.Basic/Grid; disabling split fallback adds 20%/12%. One deliberate interruption and restart preserves the result and adds about 17% wall time relative to an uninterrupted four-worker run. Three malformed-proof controls are rejected.

External projects. We check Newspeak, Solar and an upstream-described AI-generated proof collection through a relocated installation. All 126 common local roots pass both ProofServe and official default checking. ProofServe checks the full dependency closures; official default checks local roots while trusting imports. Table 2 reports their costs separately. The AI collection's model and proportions of generated or manually edited proofs are undisclosed. We also run audit on these projects; Solar has a separate EVM checking flow that is outside this experiment.

6 Discussion and limitations

What the measurements establish. The computational examples demonstrate retained evaluation, not a universal law separating two classes of proof. Compilation follows dependency order, and memory-based deferral biases the completed subset. Missing modules may be more expensive. Our P/E totals are separate hardware-specific lower bounds on the measured per-module protocol; we do not convert cores or extrapolate the tail. Call-site counts are source labels, not proof counts or dynamic kernel CPU. Whole-library rechecking did not complete within the time budget.

The short-module cohort is selected using historical build times, and its build/replay measurements are not contemporaneous randomized pairs. Observation itself can change the measured system CPU substantially: in a separate Grid no-guard control, counter profiling adds 7% wall time but 127% system CPU. We therefore keep plain timing runs separate from profiles and syscall traces; sampled cycle shares are not additive phases of those timings [8]. The memory-guard comparison keeps an outer limit but changes enforcement policy; equal outputs on passing examples do not establish identical behavior under memory exhaustion. The large/small search-path contrast has one observation per condition. We use these results to explain mechanisms, not to predict every project.

Performance and trust. We retain warm file caches and fixed run order. During release evaluation, an unrelated resident workload uses about one core on the same machine. The external-project results each have one observation, and the AI run uses bounded memory. These conditions limit generalization; the reported costs describe the actual experiments rather than an exclusively idle machine.

In addition to the logical kernel, we trust artifact loading, our interface between Lean and native code, worker coordination and locally saved results. The saved state is editable, and its noncryptographic file identities are not remote attestation. A source revision does not establish that every compiled artifact came from that source. Declared axiom inventories are not minimal per-theorem dependencies or certificates of freedom from axioms such as `sorryAx`. Strict body isolation narrows one context difference; it does not promise official parity for every malformed artifact or unsupported native evaluation.

7 Related work

Independent kernel checking and parallel proof processing precede this work. Isabelle and Coq support parallel or asynchronous checking [13, 1]. Lean provides standalone checking and a shipped `lake check` workflow [6, 5]; Nanoda supplies a separate checker [9]. ProofServe contributes an original-kernel project workflow informed by measured cost and coverage, rather than a new logical kernel.

An earlier report builds and extensively rechecks the same corpus [10]. Its 13.8-hour cloud build uses another revision and machine; its CPU estimate extrapolates a small laptop sample. We instead measure module costs and profile checking overheads. Kimina and AXLE amortize Lean preparation for verification services [12, 14].

8 Conclusion

Proof checking avoids proof search, but computational certificates can retain expensive evaluation. Our measurements show concentrated compilation costs, incomplete default-build coverage and substantial tooling overheads. ProofServe combines parallel original-kernel checking with explicit coverage reports. Together, these results show why checking cost and checking scope must be measured separately.

Artifact. Source, committed measurements and reproduction instructions are at https://github.com/yunwei37/proofserve. The paper's artifact guide maps every figure and observation to its input CSVs, tool revisions, exact commands, missing configurations and timing boundaries. It also retains the detailed compilation chronology and supplementary library-checking ledger.

References

[1] Bruno Barras, Carst Tankink, and Enrico Tassi. Asynchronous processing of coq documents: from the kernel up to the user interface, 2015.

[2] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[3] Lean contributors. Lean 4.34.1 bundled checker: module task dispatch. https://github.com/leanprover/lean4/blob/v4.34.1/src/LeanChecker.lean, 2026.

[4] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026.

[5] Lean contributors. Lean 4.35.0 release notes, 2026.

[6] Lean contributors. lean4checker and the built-in leanchecker, 2026.

[7] Linux man-pages project. getrusage(2): Linux manual page, 2026. Accessed 15 October 2026; year denotes access year.

[8] Linux perf contributors. perf-report(1): Linux manual page, 2026. Children and Overhead calculation; accessed 16 October 2026; year denotes access year, not publication date.

[9] Nanoda contributors. Nanoda: external lean type checker, 2026. Independent kernel implementation.

[10] nealmcb. Verifying openai/math in lean: cost and checker study. Technical report, revision c9570a82, 2026. Full report inspected 3 November 2026.

[11] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[12] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[13] Makarius Wenzel. Parallel proof checking in isabelle/isar. In *ACM SIGSAM Workshop on Programming Languages for Mechanized Mathematics Systems (PLMMS)*, 2009.

[14] Jimmy Xin, Alex Schneidman, Chris Cummins, Karun Ram, Srihari Ganesh, and Jannis Limperg. AXLE: A cloud infrastructure for lean 4 theorem proving utilities, 2026. Version 1; accepted at the 3rd AI for Math Workshop, ICML 2026.